

Defense Notes — agent-hands

Anticipated hard questions and honest answers

Ashish Kosana

19 August 2026

Contents

The artifact & result contract	3
Deterministic replay	4
Locators	5
Escalation & handoff	5
Safety	6
Scope	7
Escalation & handoff (as built)	7
Policy (as built)	8
Production storage	9

The walkthrough I'd give in an interview: for each load-bearing piece, the hardest questions I expect, with the honest answers. If an answer says “as designed” vs “as built”, that distinction is deliberate — inflating it would cost more than the gap.

The artifact & result contract

Q: Your discovery run succeeded, so the not-found path was never visited. Where did the MEMBER_NOT_FOUND recognizer come from, and how do you know it matches what the app actually renders — including when the message echoes the searched ID, like “No member matching ‘12345’”?

It can't come from one happy-path run. Recognizers come from two places: extra short discovery runs per declared outcome — I run discovery once with a member ID I know doesn't exist, and the terminal observation distills into the recognizer with parameter substitution applied to the pattern itself, so the echoed ID becomes a placeholder — or they're hand-authored, in which case the artifact marks them `authored` and unverified until an eval scenario exercises them, and the eval table reports the verified/unverified split. Without that, an unrecognized not-found page times out into FAILURE — exactly the outcome/failure conflation the brief warns about.

Q: The checkpoint is “the Member Details heading is visible.” What stops replay from returning the wrong member's balance as SUCCESS — say the search keypress was swallowed and the previous member's page is still showing?

In my first draft — nothing, and I'd rather admit that than defend it. The heading proves I'm on a details page, not that it's the right member, and a confident wrong answer is worse than any failure in this domain. The fix is structural: checkpoint conditions take parameter placeholders, distillation must emit at least one identity-binding condition (the member-ID cell equals `{member_id}`) whenever the capability has identifying params, publish-time validation refuses a checkpoint that references no parameter, and output extraction is anchored inside the identity-verified region.

Q: Tenant X's overlay adds an outcome code the base artifact doesn't declare. What version does a calling agent pin, and what's the outcome enum in its contract?

That would make the same name@version have a different observable contract per tenant, which defeats versioning. The rule: outcome codes are closed at the base artifact, top-level, part of the caller-facing contract. Overlays may only add recognizers mapping to already-declared codes or to internal recoverables. A genuinely new business outcome is a contract change — base version bump, all callers see it. Every trace logs the effective-artifact hash (base + overlay), so any run is attributable to the exact contract it executed.

Deterministic replay

Q: Your replay returns MEMBER_NOT_FOUND with no model in the loop. Walk me through why that classification can't be a stale page, a mid-navigation race, or the phrase "no member matches" sitting in help text.

Three properties of the condition schema: recognizers are armed per step range, they match structurally — a named region or role, never bare text anywhere in the page — and they're evaluated only against state proven fresh relative to the action (navigation observed, or the pre-action state seen gone). If a postcondition and a recognizer are simultaneously true, the postcondition wins. And every OUTCOME carries the matched node's role/region context as evidence, so an auditor verifies the classification instead of trusting it.

Q: Step 5 is "click Submit Transfer." The confirmation page is slow and the postcondition times out — can your system ever click Submit twice? Include the case where a human took over mid-run and handed back.

Structurally no, on three rules: irreversible steps are never auto-retried — on timeout they escalate; before any re-act on any step, the engine probes for the action's effect (current postconditions, then a forward scan of later steps' conditions and the checkpoint) and re-acts only if the effect is provably absent; and hand-back resumes by that same forward position-scan, not by blind current-step re-verification. If position is ambiguous it stays paused and re-escalates. Double-execution of an irreversible action has to be structurally impossible, not unlikely.

Q: You "prove" zero-LLM replay by unsetting the API key and asserting a module was never imported. What stops that passing while replay calls a model over raw HTTP?

Nothing — that check alone is name-brittle and order-dependent under pytest, which is why it's only a labeled smoke test. The real proof is hermetic: replay runs in a fresh subprocess with no API keys and all network egress blocked at

the socket level except the fixture host, asserting success plus zero model events in the trace. It doubles as a test of the Policy allowlist, which enforces the same egress boundary in production.

Locators

Q: Rung 1 is role + accessible name, and your own fixture is deliberately non-semantic. What's the accessible name of your Member ID input, and what fraction of steps does rung 1 actually carry?

Empty — a bare input in a table cell has no accessible name, and any claim otherwise overclaims. Roles are always computed; names are reliable for buttons, links, and text-bearing cells, and frequently absent for legacy form controls. That's why the ladder exists: rungs 2–4 carry form controls on genuine legacy markup, and rung 2's proximity half is a custom resolver I build, not a Playwright primitive. The semantic story survives because rungs 2–4 still anchor to what a human operator reads — label text, visible text, geometric position relative to a text anchor — which is what stays stable across tenant brandings, unlike DOM structure. And I measure rather than assert: telemetry records which rung matched per step, and the eval report shows the distribution, CSS rung expected at zero.

Q: “Exactly one element or fail” — what stops rung 4 or 5 from uniquely matching the WRONG element on a tenant it has never seen, returning a wrong balance instead of an error?

Uniqueness is not correctness — that's the sharpest version of the question. Three mitigations: every target records a fingerprint (role, editability, region) verified at replay, mismatch failing hard on risky steps; the CSS rung is disabled entirely when the replay tenant differs from the recording tenant; and extracted outputs must pass their declared parse spec or the run fails loudly. Wrong-but-confident is the one outcome this system must never produce.

Escalation & handoff

Q: Your operator takes control to fix a session expiry and, while they're in there, finishes the member search. They hand back. What does your engine do?

Resume is a forward scan, because the common case is exactly this. First, validate the current URL against the allowlist. Then check the capability checkpoint — if Member Details is showing for the right member, skip straight to output extraction. Otherwise scan forward from the current step to the

furthest consistent pre/postcondition frontier and resume there. If no unique resume point matches, re-escalate with an “I completed it / retry from here” affordance rather than guess — the same no-guess rule the ladder enforces.

Q: You promise the trace never contains secrets, and you promise to record everything the human did. The human’s fix is re-entering their password. Which promise do you break?

Neither, because capture records targets, not values: “typed 12 characters (masked) into textbox ‘Password’ in frame main” — for every field, not just type=password, because on these surfaces SSNs live in plain-text inputs and no heuristic finds them all. Screenshots are suppressed for the whole HUMAN window. The honest cost: human data entry is evidence, not a replayable macro — if the workaround should become automation, discovery re-runs through that path. And there’s a canary test: a scripted “human” types a sentinel during handoff and the suite asserts it appears nowhere under runs/.

Q: I click “Take control” while your engine is ten seconds into a thirty-second wait. Who owns the browser for the next twenty seconds?

The engine, until it acknowledges — and that’s bounded by one poll tick, not twenty seconds. The token is engine-owned; the console only posts requests (it’s an HTTP thread that never touches Playwright — the sync API is thread-affine). The engine polls the request flag on every tick of the postcondition poll loop, parks at the next tick, discards the in-flight wait’s result, and only then does the console show control as granted. The operator’s rule is simple: don’t touch until the console says it’s yours.

Safety

Q: Discovery ships page observations to an external LLM API. In production those pages show member SSNs and balances. How does discovery ever run against a real tenant?

Not casually — discovery is a privileged, attended, roughly once-per-capability event, run against UAT/training environments with synthetic records wherever they exist. Where it must touch real data: zero-data-retention terms with the provider, attended-only, and element-identity masking for values we can classify. What I won’t claim is reliable redaction of arbitrary page-native PII — you can’t redact what you can’t classify. The architectural answer is

containment: the production path — replay — sends nothing to any model, ever, and that's proven hermetically. That containment is the entire point of the artifact/replay split.

Q: Your irreversible-action labels come from the same model that's trying to finish the goal, plus keywords. What stops a mislabeled “OK” button from moving money in unattended replay?

The posture is inverted so the classifier isn't load-bearing: any action that provokes a non-GET request during discovery defaults to risky; model and keyword signals can only raise risk, never lower it; and a capability is ineligible for unattended replay until a human has reviewed the diffable step list and signed the risk labels — approval binds to the artifact hash, so re-recording invalidates it. A false negative then requires a human to miss it on an explicit sign-off, not a mid-tier model to mislabel it silently.

Scope

Q: You have six slices, one timebox — and you're also building the target app, a second skin, an operator console, and an eval harness. If you lose two days, what ships and which must-have dies first?

No must-have dies; scope leaves in a published order: the saucedemo portability leg first, the Lakeside skin second (tenancy falls back to overlay-merge unit tests plus the design — and never both, that would leave zero generalization evidence), the operator console degrades from web page to CLI on the same state machine third, the drift eval case fourth. Never cut: the artifact schema, replay engine, discovery, the three-way taxonomy, the escalation state machine, and the zero-LLM proof. Slice order was chosen so the highest-weighted deliverable — the replay contract — exists and stays runnable from day one.

Escalation & handoff (as built)

Q: Your operator console is a bare HTML page with five buttons. Is that the handoff the brief asked for?

The brief explicitly scopes the console out (“mock the operator UI if needed — make the handoff mechanism and the control-transfer model real”). What's real here: an engine-owned token with request/acknowledge semantics so two drivers on one session is impossible; interventions carrying step, reason, recent trace, params, and a screenshot; four attributed exits (take/hand back, approve, abort, resolve-as-declared-outcome); TTL expiry that closes the

session; capture that records semantic descriptors with masked lengths; and a resume that position-scans forward because humans finish the screen they're on. The evidence folder has a live captured handoff ending in SUCCESS.

Q: Your canary test — did it ever actually catch anything?

Yes, during development: the first version leaked the sentinel through the FAILURE evidence path — an operator aborted, the engine captured an accessibility snapshot, and the human's typed text was still sitting in a field on the page. That's exactly the class of leak string-matching can't find. The fix suppresses page-content evidence once a human has driven the session; the refinement (element-level taint masking instead of suppression) is stated as next work. I'd rather show the canary catching a real bug than claim the redaction was right the first time.

Policy (as built)

Q: What stops a capability from wiring money unattended?

Three independent layers. The network allowlist means it can't even fetch beyond the entry host. The mutating-by-default rule means any step that fired a non-GET during discovery is recorded risky — observed through the page's reaction, because the POST usually fires after the driver call returns; that race was found in review and is now regression-tested. And risky steps refuse unattended replay unless a human signed the artifact, with the signature bound to the artifact hash — re-record it, or edit one step, and approval is void. The failure mode requires a human to miss it on an explicit sign-off, not a model to mislabel it silently.

Q: You cut the second tenant AND the public-site leg — your own design said never both. Defend that.

Guilty, and the report says so in those words. With a fixed budget the choice was between a second self-authored skin and depth on escalation, policy, and measured evals — which are the three highest-weighted criteria I hadn't yet proven. What stands in for generalization evidence: the drift eval (renamed label → loud failure listing every rung tried), per-step rung telemetry in every trace, and a schema with no web-specific field outside one flagged rung. If the interview wants it demonstrated, the overlay loader plus a divergent skin is the first item on the next-steps list, and the schema was shaped so that work is additive.

Production storage

Q: Where do these JSON artifacts live in production?

In git — deliberately, because an artifact has source code's lifecycle, not a database row's: it's reviewed (the risk sign-off becomes a pull request), versioned (callers pin name@version), diffable, and promoted. CI runs the schema's load-time validators on merge. In front of that, a thin registry: (name, version, tenant) resolving to a content hash, blobs in object storage keyed by sha256 so nothing changes under a caller silently — every trace already logs that hash, so an auditor can go from any run to the exact bytes of the contract that executed. What does NOT go in git: per-run traces (object storage with retention), drift/health telemetry, and promotion state (database, keyed by artifact hash). The rule: immutable-and-reviewed lives in git and content-addressed blobs; mutable-and-per-run lives in a database. Putting everything in Postgres works mechanically but you'd rebuild what git gives free — diff review, immutable history, rollback, blame. The repo's files-in-git layout isn't a shortcut; it's this design scaled down.